



AI with Rav

Learn AI the way you actually think.



DAY 6 of 30

MACHINE LEARNING

Logistic Regression — the yes/no brain

WHAT YOU'LL LEARN TODAY

- › When to use Logistic vs Linear Regression
- › The sigmoid S-curve + its formula
- › Why it's called "regression" (the surprise answer)
- › How the 0.5 threshold turns probability into YES/NO



Logistic Regression — the yes/no brain

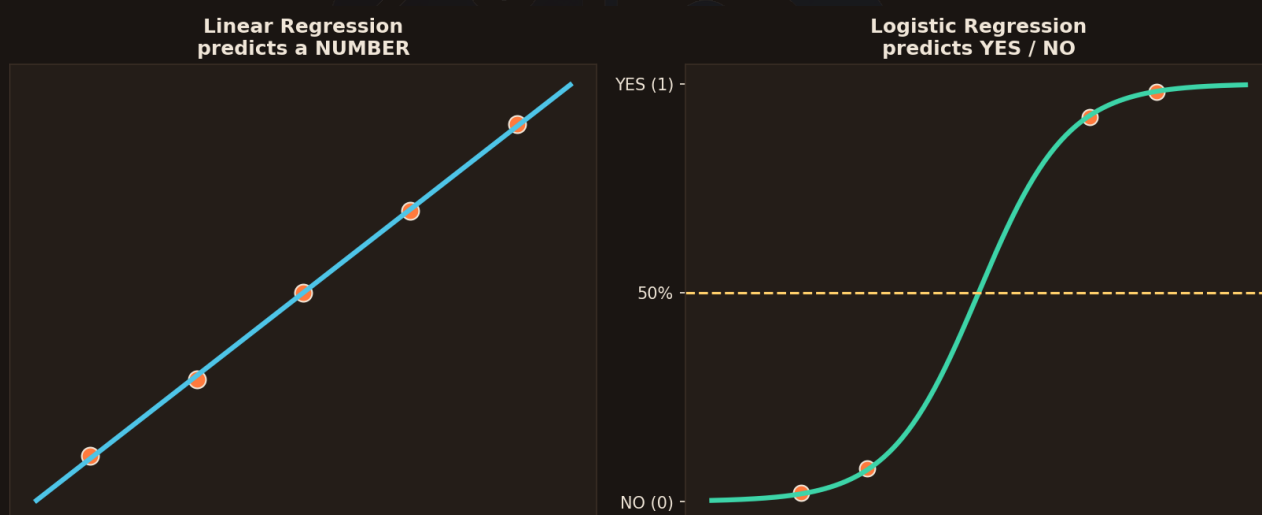
In One Line: Logistic Regression answers YES/NO questions. Spam or not? Pass or fail? Fraud or safe? It gives a probability, then a threshold decides.

Start here: the watchman at the gate

A society watchman looks at each person walking up: known face, calm, has a delivery bag → **let in**. Unknown, nervous, no reason → **stop**. He's not predicting a *number* — he's making a **yes/no** call from clues.

Yesterday's Linear Regression predicted a **number** (a price). But tons of real problems are **yes or no**: is this email spam? will this customer leave? is this transaction fraud? For those, you need **Logistic Regression** — the yes/no brain.

Linear vs Logistic — the one difference



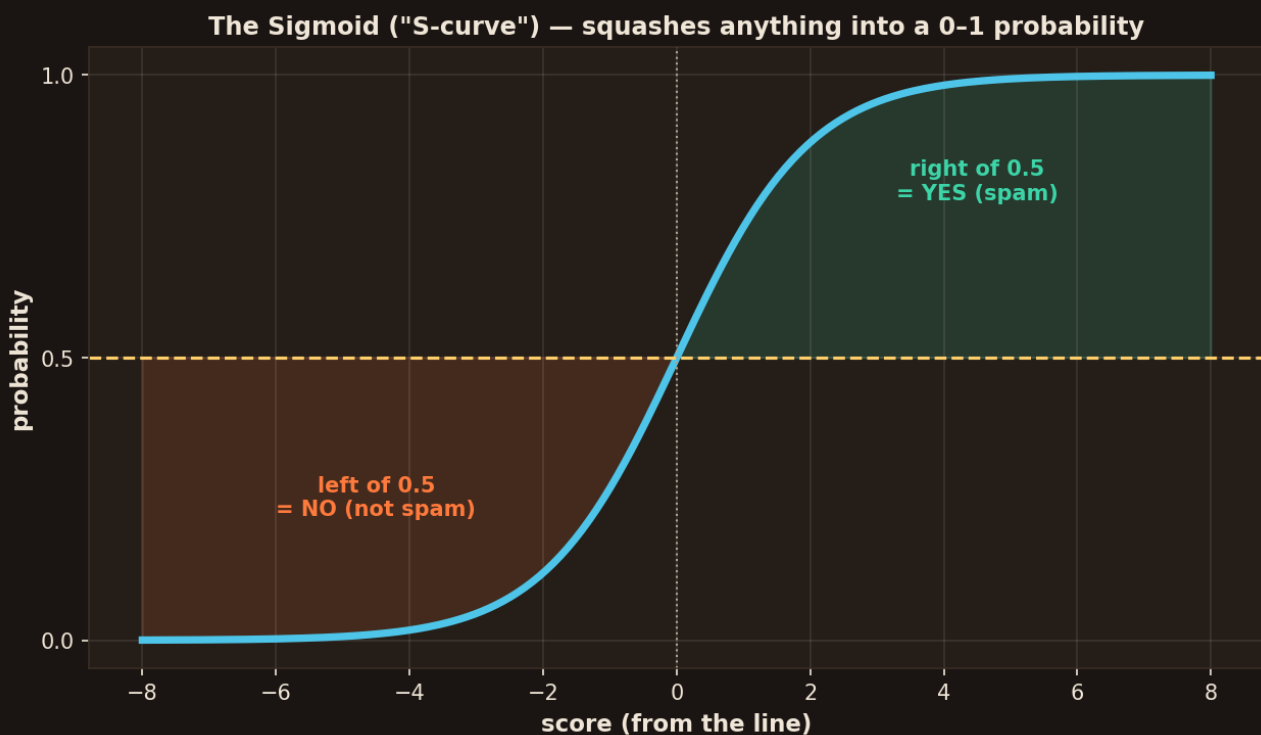
Linear Regression fits a line to predict a number. Logistic Regression fits an S-curve to predict YES or NO.

- **Linear Regression** → answer is a **number** (price, sales, temperature). Straight line.
- **Logistic Regression** → answer is **yes/no** (spam, fraud, pass/fail). S-curve.

Quick test: "Am I predicting a number, or a yes/no?" Number → Linear. Yes/No (a category) → Logistic. That single question tells you which one to reach for.

The magic: the S-curve (sigmoid)





The sigmoid squashes ANY score into a probability between 0 and 1. Right of the middle = likely YES, left = likely NO.

A straight line can shoot off to any number — but a probability must stay between **0 and 1**. So Logistic Regression runs the line's score through a special **S-shaped curve** (the *sigmoid*) that squashes anything into a 0–1 probability.

- A very "spammy" email → score is high → sigmoid pushes it near **1** (almost certainly spam)
- A clearly normal email → score is low → sigmoid pushes it near **0** (not spam)

The sigmoid is the whole trick. It turns "how spammy is this?" (a raw score) into "what's the probability it's spam?" (a clean number between 0 and 1).

The formula (for the curious — skip if you like)



The Sigmoid formula — behind the S-curve

$$P = \frac{1}{1 + e^{-z}}$$

probability (0 to 1)

$e = 2.718...$
(just a constant)

$z = m \cdot x + c$
(the SAME line from Day 5!)

It takes the straight line $z = mx + c$ and squashes it into a probability between 0 and 1.

The sigmoid formula. Notice $z = mx + c$ is the exact same line from Day 5 — the sigmoid just wraps around it.

Here's the actual formula behind the S-curve. Don't be scared — it's simpler than it looks:

- $z = m \cdot x + c$ → this is literally the **same straight line from Day 5** (Linear Regression's line).
- $P = 1 / (1 + e^{(-z)})$ → we feed that line's score `z` into this, and out comes a probability `P` between 0 and 1.
- e → just a fixed number (2.718...), like π . Nothing to memorise.

The big idea: Logistic Regression = Linear Regression's straight line, wrapped in a sigmoid. Same ` $mx + c$ `, then squashed to a 0–1 probability. That's the entire difference. Technical folks: this is why it's linear "under the hood." Non-technical folks: just remember — it's the same line, made into a yes/no.

Wait — why is it called "regression" if it's yes/no?

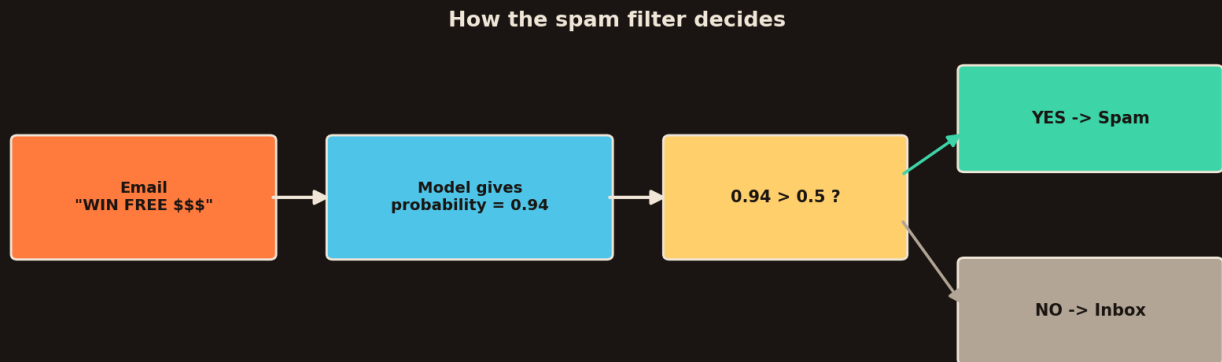
Great question, and almost no course explains it. "Regression" usually means predicting a number — so why call a yes/no method "regression"?

- Because under the hood, it **first predicts a number** — the probability (like 0.94). That part IS regression (predicting a number between 0 and 1).
- Only at the **very end** does the threshold turn that number into yes/no.
- So the name comes from the *middle* step: it **regresses a probability**, then classifies. The classification is bolted on top.

One-line answer: it's called regression because it predicts a **probability** (a number) first — the yes/no decision is just the final step on top of that. So it's really "probability regression, then a cutoff."



The threshold: turning probability into a decision



The 0.5 line is the "threshold" — you can move it to be stricter or looser.

The model gives a probability; the 0.5 threshold turns it into a final YES or NO.

The model gives a probability like **0.94**. To make a real decision, you pick a **threshold** — usually 0.5:

- probability **above 0.5** → **YES** (spam → junk folder)
- probability **below 0.5** → **NO** (safe → inbox)

You can MOVE the threshold. For fraud, you might use 0.3 (catch more, even with false alarms). For blocking someone, maybe 0.8 (only when very sure). The threshold is your control dial.

Train a spam classifier — a few lines

```
from sklearn.linear_model import LogisticRegression

# X = features (e.g. num of $ signs, CAPS words, links)
# y = 1 for spam, 0 for not-spam
model = LogisticRegression()
model.fit(X_train, y_train)

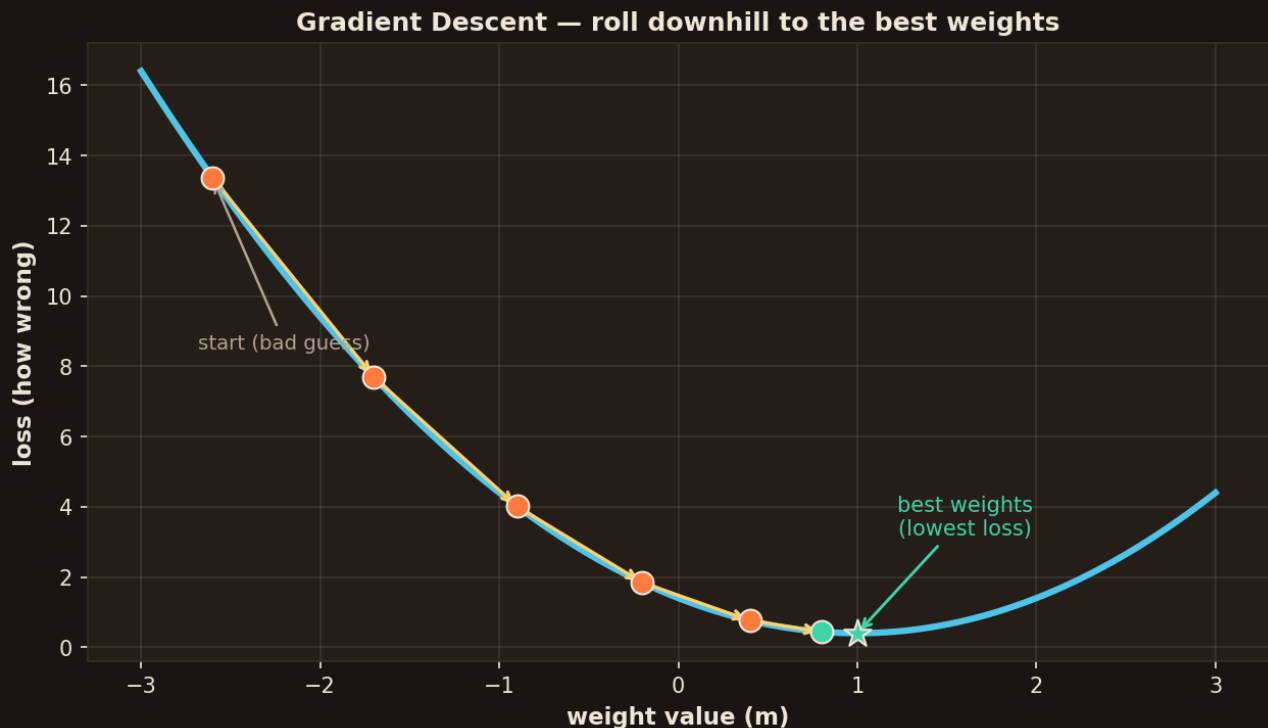
print(model.predict([[5, 8, 3]]))      # -> [1] (spam)
print(model.predict_proba([[5, 8, 3]])) # -> [[0.06, 0.94]] 94% spam
```

``predict`` gives the yes/no. ``predict_proba`` gives the probability behind it — that 0.94 is the sigmoid's output. Same 5-line pattern as Linear Regression, just a different answer type.



■ Under the Hood — how it actually learns (technical, skip if you like)

Non-technical folks: you've got the whole idea already — jump to the recap. Technical folks: here's the real engine.



Training = gradient descent. Start with random weights (high loss), then step downhill until the loss is lowest. That's how m and c are found.

- **The loss function** → Logistic Regression doesn't use MSE. It uses **log-loss** (cross-entropy): $-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$. It punishes confident-but-wrong predictions harshly (predict 0.99 when the truth is 0 → huge loss).
- **How it finds the weights** → **gradient descent** (the picture above): start with random m , c , compute the loss, nudge the weights in the direction that lowers it, repeat thousands of times until it settles at the minimum.
- **Why not a straight line's MSE?** → with the sigmoid, MSE gives a bumpy loss surface with local traps. Log-loss makes it a clean bowl, so gradient descent reliably reaches the bottom.

When it breaks (what an engineer must know): (1) **Class imbalance** — 99% not-spam, 1% spam → the model can score 99% "accuracy" by always saying "not spam" and be useless. Use precision/recall, not accuracy (Video 17). (2) **Non-linear boundaries** — logistic draws a straight decision line; if your classes curve around each other, it fails — you need features engineered or a tree/NN. (3) **Correlated features** (multicollinearity) make the learned weights unstable and hard to trust.



Recap — the 20-second version

- Logistic Regression answers **yes/no** questions (spam, fraud, churn).
- Use it when the answer is a **category**, not a number.
- The **sigmoid (S-curve)** turns any score into a 0–1 **probability**.
- A **threshold** (usually 0.5) turns the probability into the final YES/NO.
- You can move the threshold to be stricter or looser.

Next up — Video 7: Decision Trees. The "20 Questions" game that can classify almost anything — and the easiest algorithm to actually see thinking. See you Day 7.

